

Designing, Building and Verifying an Embedded System

Introduction to ARM Cortex-M Microprocessors – Chapter 14

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1 **Why This Matters**

- 2 **From Requirement to Architecture**

- 3 **The Non-Blocking Main Loop**

- 4 **Verification**

- 5 **Design Challenge**

- 1 **Why This Matters**
- 2 From Requirement to Architecture
- 3 The Non-Blocking Main Loop
- 4 Verification
- 5 Design Challenge

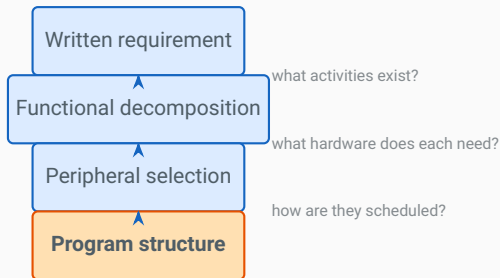
Every earlier chapter looked at one peripheral by itself. Real systems do not work that way. A real system samples on a schedule. It responds to buttons. It drives indicators. It reports to a host. It does all of this at once, on one processor, with no operating system. The hard part is not the peripherals, and it is not the code. The hard part is deciding, before any code exists, which peripherals the job needs and whether the timing can work at all. Then you must *prove* the result meets the requirement, not just that it seems to work. This chapter does all three steps, in the order an engineer actually works.

After completing this chapter, you should be able to:

1. Turn a written requirement into a functional decomposition and a peripheral selection.
2. Choose between polling, timed polling, and interrupt-driven structures for a given activity.
3. Structure a bare-metal program using a non-blocking cooperative main loop.
4. Identify and protect shared data between interrupt and main contexts.
5. Produce a timing budget before implementation and verify it by measurement afterwards.
6. Apply a systematic bottom-up procedure to diagnose a multi-peripheral system.

- 1 Why This Matters
- 2 **From Requirement to Architecture**
- 3 The Non-Blocking Main Loop
- 4 Verification
- 5 Design Challenge

A requirement is prose. An architecture is a set of decisions. Getting from one to the other is **the engineering**, and it happens on paper.



Why this order matters

Choosing peripherals before decomposing the activities means you discover the pin conflict **after** the board is made.

Four Ways to Structure an Activity

Structure	Use when	Cost
Blocking poll	Never, in a real system	Burns the CPU; blocks everything else
Timed poll	The activity has a known period and loose jitter	Simple; jitter is the loop time
Interrupt	The event is external and the deadline is tight	Needs care with shared data
DMA	Bulk data, no per-byte decision	Setup cost; fewest CPU cycles

The decision is per activity, not per program

One system routinely uses all four: DMA for an ADC stream, an interrupt for a button, a timed poll for a display refresh, and the main loop for everything with no deadline at all.

- 1 Why This Matters
- 2 From Requirement to Architecture
- 3 **The Non-Blocking Main Loop**
- 4 Verification
- 5 Design Challenge

```
while (1) {  
    uint32_t now = millis();  
  
    if (now - last_sensor >= 2000) {    // every 2 s  
        last_sensor = now;  
        read_sensor();  
    }  
    if (now - last_display >= 100) {    // every 100 ms  
        last_display = now;  
        update_display();  
    }  
    if (button_flag) {                  // set by an ISR  
        button_flag = 0;  
        handle_button();  
    }  
}
```

Nothing waits

Each activity asks “is it my turn yet?” and returns immediately if not. The loop runs thousands of times per second, and every activity gets serviced **close enough** to its deadline.

Why `now - last >= period` and Not `now >= next`

The subtraction form

```
if (now - last >= 2000)
```

Survives the 32-bit counter **wrapping** after 49 days, because unsigned subtraction wraps consistently.

The comparison form

```
if (now >= next)
```

Breaks at the wrap: `next` is a huge number, `now` has reset to a small one, and the activity stops for 49 days.

A bug you cannot find by testing

It appears once, after seven weeks of continuous running. This is exactly the class of fault that **reasoning** catches and **testing** does not — which is why the argument matters more than the code.

Two hazards

- **Missing volatile** — the optimiser caches a value the ISR changes
- **Torn access** — `main()` reads a multi-word variable while the ISR is halfway through writing it

Three defences

- Mark it volatile
- Keep it to a single `uint32_t` — read atomically on a 32-bit machine
- Or disable interrupts **briefly** around the access

The flag pattern

The ISR does the minimum — clear the hardware flag, set a software one, return. `main()` does the real work. That keeps the handler short **and** keeps the shared state to one byte.

- 1 Why This Matters
- 2 From Requirement to Architecture
- 3 The Non-Blocking Main Loop
- 4 **Verification**
- 5 Design Challenge

Write it **before** implementation, and it tells you whether the design is possible at all.

Activity	Period	Duration	Load
SysTick ISR	1 ms	2 μ s	0.2 %
Sensor read	2 s	400 μ s	0.02 %
Display update	100 ms	3 ms	3 %
Button ISR	rare	5 μ s	$\approx$ 0
Total			$\approx$ 3.3 %

What the number tells you

3 % means comfortable headroom. **80 % means the design is fragile** – one slow path and a deadline is missed. Over 100 % means it cannot work, and you know that before writing a line.

The discipline

1. **Predict** the number — period, duty cycle, execution time
2. **Measure** it
3. When they disagree, **find out why**

How to measure

- Set a GPIO pin high on entry, low on exit — watch it on a scope
- Read `SysTick->VAL` before and after
- Toggle a pin per loop iteration and measure the frequency

The disagreement is the finding

If you predicted 400 μ s and measure 3 ms, you have learned something — about the compiler, the clock tree, or your own assumptions.

Measuring without predicting teaches you nothing, because any number looks plausible.

It runs. What is still unknown?

- Whether it survives the **counter wrap** at 49 days
- Whether the deadline is met in the **worst case**, not the typical one
- Whether a shared variable is ever **torn**
- What happens when a sensor **fails** rather than reads wrong
- Whether it works at the **temperature extremes**

Demonstration is evidence, not proof

A demo shows the system works **once, here, now**, under the conditions you happened to create.

Everything else you believe about it comes from **argument**: the timing budget, the wrap analysis, the reasoning about shared state.

When a multi-peripheral system misbehaves, work **bottom-up**. Each step either finds the fault or eliminates a layer.

Step	Question
1	Is the clock enabled for this peripheral?
2	Is the pin in the right mode, with the right alternate function?
3	Does the register read back what you wrote?
4	Does the peripheral's status flag ever set?
5	Does the ISR run — toggle a pin inside it and look
6	Is the shared variable <i>volatile</i> , and read atomically?
7	Does the timing match the budget you predicted?

Steps 1 and 2 account for most first-time faults — and **a register that does not read back is an unlocked peripheral**.

The mechanism

- A CPU fetches, decodes, executes — forever
- Memory is numbered; some numbers are hardware
- C reaches them through **pointers**
- Instructions are what your C becomes

The practice

- Enable the clock first
- **volatile** on everything hardware touches
- Read–modify–write preserves the neighbours
- Interrupts for deadlines, timers for time
- **Predict, then measure**

The one sentence from Chapter 1

The CPU has no special instruction for “turn on an LED”. It has instructions that read and write addresses — and some of those addresses are wired to hardware.

Everything in the thirteen chapters since has been an elaboration of that.

- 1 Why This Matters
- 2 From Requirement to Architecture
- 3 The Non-Blocking Main Loop
- 4 Verification
- 5 Design Challenge

The brief

A greenhouse controller must do the following: read a temperature sensor every 2 s; read a soil-moisture sensor every 30 s; drive a ventilation fan through a PWM output whose duty cycle follows temperature; open a water valve (a single digital output) for 5 s whenever moisture drops below a threshold; accept a manual override button at any time; and report all readings to a host once per minute.

Do not write any code. Produce the design: the functional decomposition, the peripheral assigned to each activity, the event structure chosen for each (with reasons), a pin allocation checked for collisions on the STM32F446RE, a timing budget, and the initialization order. Then write the verification plan that goes with it: what measurement proves each timing requirement, with what instrument, and what the pass/fail criterion is. Identify the one activity whose timing is hardest to guarantee, and the single measurement you would take first.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 14